

Kaggle Challenge: IEEE-CIS Fraud Detection

Salam AL-KAISSI, Mayra SUAREZ, Le HA, Junzhang ZHONG

March 2025

Abstract

This report summarizes the approach and findings from the IEEE-CIS Fraud Detection Kaggle competition. The goal was to develop a predictive model to identify fraudulent transactions using a large dataset of credit card transactions.

1 Business Understanding

1.1 Business Objective

The primary goal of detecting fraudulent transactions can be summarized as follows:

- After identifying a fraudulent transaction, the company must immediately block the associated card.
- The system should be capable of estimating the likelihood of a transaction being fraudulent.
- It is crucial to minimize both false negatives (fraudulent transactions misclassified as legitimate) and false positives (legitimate transactions flagged as fraud). Therefore, maintaining a balance between precision and recall is essential.

1.2 Situation Assessment

Fraud detection is critical in financial services, where losses can be significant. Understanding transaction patterns and identifying anomalies is essential for effective fraud prevention.

1.3 Data Mining Goals

- Develop a robust model to classify transactions as fraudulent or legitimate. - Achieve high accuracy while minimizing false positives to avoid customer inconvenience.

2 Data Understanding

2.1 Data Collection

The dataset consists of millions of transactions, including features such as transaction amount, time, user demographics, and transaction type.

2.2 Data Description

There are four files: *train transactions*, *train identity*, *test transactions*, and *test identity*.

- These components are linked using the **TransactionID** field.
- The target feature, *isFraud*, is a binary variable indicating fraud status.
- Not all transactions have corresponding identity information.

2.2.1 Transaction Data

The transaction dataset contains various details related to each payment, including:

- **TransactionID**: Unique identifier for each transaction.
- **TransactionDT**: Time offset from a reference point (not an actual timestamp).
- **TransactionAMT**: Transaction amount in USD.
- **ProductCD**: Categorical variable representing the product type for the transaction.
- **Card1 - Card6**: Information related to the payment card, such as type, category, and issuing country.
- **Addr1, Addr2**: Address-related details, potentially indicating billing region.
- **Dist1, Dist2**: Certain distance-based variables.
- **P_emaildomain**: Email domain of the purchaser.
- **R_emaildomain**: Email domain of the recipient.
- **C1 - C14**: Count-based features, which may indicate the number of times an address or card has appeared in transactions. The precise meaning is obfuscated.
- **D1 - D15**: Time difference variables, likely representing intervals between transactions.
- **M1 - M9**: Categorical indicators, potentially signaling data matches such as name and billing address consistency.
- **Vxxx**: A set of engineered features related to rankings, frequencies, and other transactional characteristics.

2.2.2 Identity Data

The identity dataset provides additional details about transactions when available, including:

- **TransactionID**: Unique identifier for each transaction.
- **DeviceType**: Categorical variable indicating whether the transaction was conducted via a mobile device or a desktop.
- **DeviceInfo**: Additional details about the device model and browser used.
- **ID1 - ID38**: A mix of categorical and numerical fields containing information on network connections, browser specifics, and other metadata. (Note: ID12 - ID38 are categorical).

2.3 Data Exploration

We have a training dataset of shape $(590,540, 434)$ and a test dataset of the same dimensions. Before building a model, we begin by understanding the data distribution and missing values in both sets.

As expected, the target variable, *isFraud*, is highly imbalanced, with 96.5% of transactions labeled as non-fraudulent and only **3.5%** as fraudulent. Given this imbalance, we will use Area Under the ROC Curve (AUC) as our evaluation metric to ensure the model can distinguish between fraudulent and non-fraudulent transactions effectively.

To improve performance, we will explore feature engineering techniques, iteratively selecting the most relevant features using a forward selection approach. The goal is to build a strong baseline model and refine it until a satisfactory AUC score is achieved.

- Assess data quality by checking for missing values, duplicates, and inconsistencies.
- We will see columns with more than 90% values missing.

To improve performance, we will explore feature engineering techniques, iteratively selecting the most relevant features using a forward selection approach. The goal is to build a strong baseline model and refine it until a satisfactory AUC score is achieved.

Assess data quality by checking for missing values, duplicates, and inconsistencies.

We will see columns with more than 90% values missing.

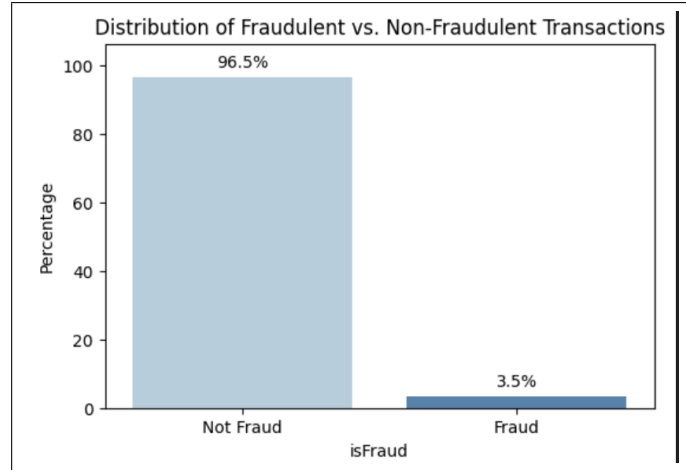


Figure 1: Distribution of Fraudulent vs. Non-Fraudulent Transactions

3 Data Preparation

3.1 Data Selection

- **Transaction Data:** Features such as `TransactionAmt`, `ProductCD`, `card1` to `card6`, and `addr1` were selected to capture transaction details.
- **Identity Data:** Features like `DeviceType`, `DeviceInfo`, and email domains (`P_emaildomain`, `R_emaildomain`) were included to capture user-specific information.
- **Temporal Features:** `TransactionDT` was used to extract temporal features like hour and day of the transaction.
- **Engineered Features:** Features like `C1` to `C14`, `D1` to `D15`, and `M1` to `M9` were included based on their potential to capture underlying patterns in the data.

3.2 Data Cleaning

- **Handling Missing Values:**
 - **Missing Value Analysis**
 - * A function `detectmissing` is defined to calculate and display missing percentages and the number of unique values for each column in a `DataFrame`.
 - * This function is applied to the merged training data to create `train_missing`.
 - * Features with more than 70% missing values in the training set are identified and listed [5-8]. A significant number of 'V' (presumably Vesta) features, 'D' (timedelta) features, and identification features fall into this category .
 - * The percentage of features with more than certain thresholds of missing values (0%, 10%, 20%, 40%, 70%, 80%, 90%, 99%) is printed for both the train and test sets.
 - **Imputation:** Missing values in numerical features were imputed using the median or mean of the respective feature. Categorical missing values were filled with a placeholder like -999.
 - **Removal of Outliers:** Outliers in features like `TransactionAmt`, `dist1`, and `dist2` were identified and capped or removed to prevent skewing the model.
- **Duplicate Removal:** Duplicate records were identified and removed to ensure the dataset's integrity.

3.3 Data Construction

- **Feature Engineering:**
 - **Temporal Features:** Extracted hour and day from `TransactionDT` to capture temporal patterns.

- **Temporal Analysis (TransactionDT)**
 - * The time span of the train, test, and total datasets, as well as the time gap between the train and test sets, are calculated based on the `TransactionDT` feature. The train and test datasets span approximately 182 and 183 days, respectively, with a gap of around 30 days between them .
 - * The distribution of `TransactionDT` in both train and test sets is visualised using histograms.
 - * New features 'day' and 'hour' are derived from `TransactionDT`.
 - * The percentage of fraudulent and legitimate transactions for each day of the week (derived) is visualised.
 - * The fraction of fraudulent transactions and the number of total transactions per hour are plotted.
 - * A new categorical feature `hour_warning` is created based on the hour of the day.
- **Aggregated Features:** Created features like `TransactionAmt_to_mean_card1` to capture the transaction amount relative to the card's average transaction amount.
- **Categorical Encoding:** Encoded categorical variables like `ProductCD` and email domains using label encoding to convert them into a numerical format.
- **Interaction Features:** Created interaction terms between transaction amounts and card types to capture complex relationships.
- **Feature Exploration and Outlier Analysis**
 - * A function `describe` is defined to provide descriptive statistics for a given feature in the train (overall, fraud, legitimate) and test sets.
 - * Descriptive statistics for `TransactionAmt` are printed for both train and test sets. Outliers with `TransactionAmt` greater than 10000 are identified and dropped from the training set.
 - * The distribution of `TransactionAmt` and its log-transformed version (`LogTransactionAmt`) are visualised for both fraudulent and legitimate transactions in the train set, and for all transactions in the test set.
 - * The distribution of `ProductCD` across fraudulent and legitimate transactions is examined.
 - * Similar explorations are conducted for `card1`, `card2`, `card3`, `card4`, `card5`, and `card6`, including descriptive statistics and visualisations of their distributions for fraud and legitimate cases.
 - * Rare categories in `card6` ('charge card', 'debit or credit') are replaced with 'debit'.
 - * The distributions of `addr1` (Purchaser Region) and `addr2` (Purchaser Billing Country) are visualised, and the number of unique values and top occurring values are printed. The majority of `addr2` values are '87.0', likely representing the United States.
 - * The distributions of `dist1` and `dist2` are examined, with outliers greater than 6000 in `dist1` and 8000 in `dist2` being dropped from the training set.
 - * The presence of `id_12`, `id_30`, `id_31`, and `id_33` columns in both train and test sets is confirmed.
 - * The distribution of `P_emaildomain` (Purchaser Email Domain) and `R_emaildomain` (Recipient Email Domain) is analysed, and domains are grouped into broader categories ('Google', 'Yahoo', 'Microsoft', 'Others') and NaN values are replaced with 'NoInf'.
 - * The continuous 'C' features (C1-C14) are explored for outliers using scatter plots against `TransactionDT`. Outliers in several 'C' features are observed around a specific time range, and these are subsequently removed from the training data [60-63]. Descriptive statistics for the 'C' features after outlier removal are presented [64, 65].
 - * The 'D' (timedelta) features (D1-D15) are examined. Many 'D' features have a high percentage of missing values, and their relationship with `TransactionDT` is visualised. Most 'D' features show a monotonically increasing trend with time.
 - * The categorical 'M' features (M1-M9) are analysed for their distribution across fraudulent and legitimate transactions using bar plots.
 - * The correlation between 'V' features (V1-V339) is visualised using a heatmap on a subset of the training data .
 - * The 'id' features (`id_01` - `id_38`) are briefly examined, noting the number of unique categories in each . The distributions of some categorical 'id' features (`id_03`, `id_04`, `id_12`, etc.) across fraud and legitimate transactions are visualised.
 - * The object type 'id' features (`id_30`, `id_31`, `id_33`) are further explored. `id_30` (Device info?) is split into 'OS' and 'version'. `id_31` (Browser info) is split into 'browser' and 'version'. `id_33` (Device Resolution) is split into 'width' and 'height'.

- * **DeviceType** and **DeviceInfo** are analysed. The distribution of **DeviceType** (desktop/mobile) across fraud and legitimate transactions is visualised. A new feature **DeviceCorp** is engineered from **DeviceInfo** by grouping similar devices. The distribution of this new feature across fraud and legitimate transactions is visualised.
- **Principal Component Analysis (PCA):**
 - * **Purpose:** PCA was applied to the **V** features to reduce dimensionality while retaining most of the variance in the data. This helps in managing the complexity of the dataset and improving computational efficiency.
 - * **Implementation:** The **V** features were scaled using min-max scaling before applying PCA. The number of components was chosen based on the cumulative explained variance, with 30 components explaining approximately 98.6% of the variance.
 - * **Outcome:** The PCA-transformed features were integrated into the dataset, replacing the original **V** features, thereby simplifying the model without significant loss of information.

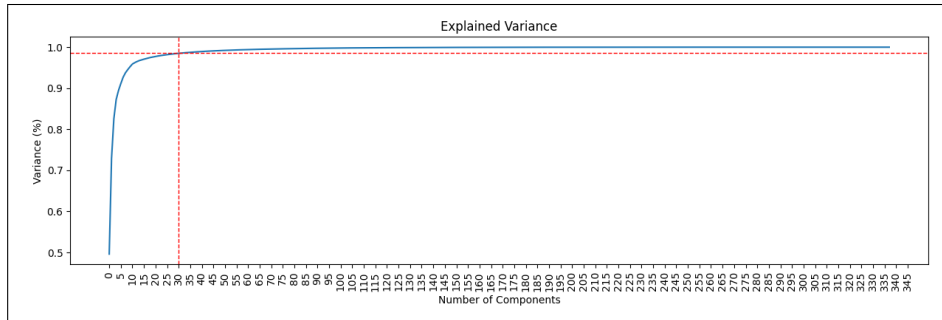


Figure 2: 30 of components can explain around 98.6 percentage of variance

3.4 Data Integration

- **Merging Datasets:** Transaction and identity datasets were merged on **TransactionID** to create a unified dataset containing all relevant features for modeling.

3.5 Data Formatting

- **Numerical Encoding:** Categorical variables were encoded using label encoding to convert them into numerical format.
- **Scaling:** Features were scaled using min-max scaling to normalize the data, ensuring that all features contribute equally to the model's performance.

4 Modeling

4.1 Modeling Technique Selection

- **Algorithms Evaluated:**
 - * **Decision Trees:** Used as a baseline model to establish a performance benchmark.
 - * **Random Forests:** Evaluated for their ability to handle complex interactions and reduce overfitting.
 - * **Gradient Boosting:** Considered for its effectiveness in handling tabular data and capturing complex patterns.
 - * **LightGBM:** Chosen as the primary algorithm due to its efficiency in handling large datasets and its ability to manage categorical features natively.
 - * **CatBoost:** Chosen because of a higher performance in algorithm accuracy and its ability to deal with categorical NaN.

4.2 Model Building

- Necessary libraries for modeling (scikit-learn, LightGBM) are imported.
- A function `reduce_mem_usage2` is defined and applied to reduce the memory usage of the train and test DataFrames by downcasting data types.
- Additional feature engineering is performed, creating features based on the mean and standard deviation of `TransactionAmt` grouped by different features like overall and by `card1` and `card4`.
- Label Encoding is applied to the categorical features.
- Principal Component Analysis (PCA) is used to reduce the dimensionality of the 'V' features, aiming to retain 98.6% of the variance with 30 components.
- The train and test sets are separated again after feature engineering.
- The target variable `isFraud` and feature matrix X are defined.
- Missing values in X and test are imputed with -999, and infinite values are replaced with NaN.
- The data is split into training and testing sets.
- A base LightGBM classifier with balanced class weights is trained, and its performance (AUC score) on the train and test sets is evaluated.
- Hyperparameter tuning for the LightGBM model is performed using RandomizedSearchCV with stratified k-fold cross-validation.
- The best hyperparameters found by RandomizedSearchCV are used to train a final LightGBM model, and its performance on the train and test sets is evaluated (AUC score). The ROC curves for both the base and tuned LightGBM models are visualised.
- Feature importance scores from the best LightGBM model are calculated and visualised.
- Predictions on the test set are made using the tuned LightGBM model, and a submission file (`submission.csv`) is created with the predicted probabilities of fraud.
- To apply the CatBoost model, only the TransactionID column is removed. All the floating NaNs are transformed into string 'missing', then we start to train a CatBoostClassifier.

4.3 Model Assessment

- **Evaluation Metrics:**
 - * **ROC-AUC:** Used as the primary metric to evaluate model performance due to its effectiveness in handling imbalanced datasets.
 - * **Additional Metrics:** Accuracy, precision, recall, and F1-score were also calculated to provide a comprehensive evaluation of model performance.
- **Model Comparison:** Different models and hyperparameter settings were compared, with the best model achieving an ROC-AUC of 0.946 on the training set and 0.926 on the test set.
- **Feature Importance:** Analyzed to identify the most influential features, with transaction amount and engineered features showing significant importance.

5 Evaluation

The primary metric for evaluating model performance was the **Area Under the ROC Curve (AUC)**. This was chosen due to the **high class imbalance in the isFraud target variable**, with only 3.5% of transactions being fraudulent. AUC assesses the model's ability to distinguish between fraudulent and non-fraudulent transactions across various thresholds.

In addition to AUC, **accuracy, precision, recall, and F1-score** were also calculated to provide a comprehensive evaluation. The importance of balancing precision and recall in fraud detection was highlighted:

- **High precision** is crucial to minimise false positives, preventing unnecessary blocking of legitimate user cards.
- **High recall** is important to identify as many fraudulent transactions as possible, reducing financial losses.

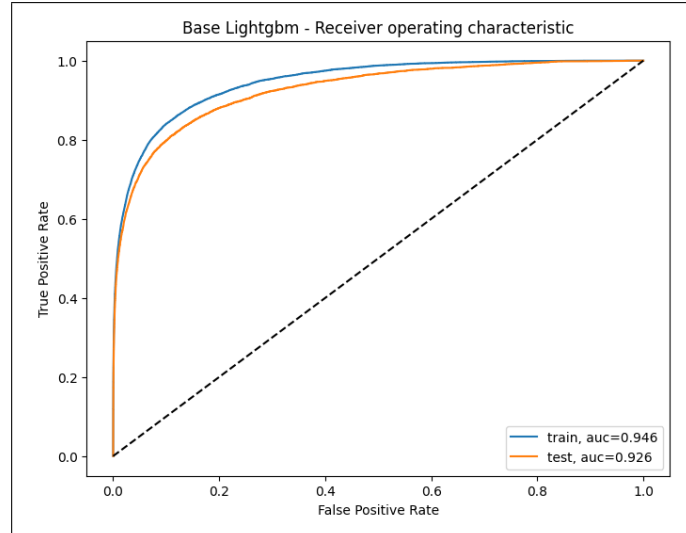


Figure 3

- The **F1-score** serves as a comprehensive measure by considering both precision and recall.

The best-performing model, a tuned **LightGBM classifier**, achieved an **ROC-AUC of 0.946 on the training set and 0.926 on the test set**. The code also demonstrates high AUC scores for both base and tuned LightGBM models. ROC curves were used to visualise the model's performance.

Feature importance analysis indicated that **transaction amounts and engineered features** (temporal and aggregated features) were significant contributors to the model's predictive power.

The Kaggle score of the model of **CatBoost: 0.892948 for private score and 0.927335 for public score**.

6 Conclusion

The study concluded that the **LightGBM model demonstrated strong performance in fraud detection**, underscoring the effectiveness of gradient boosting techniques for imbalanced datasets. A key finding was the significant contribution of **feature engineering, especially the creation of temporal and aggregated features**, in enhancing the model's ability to detect fraudulent transactions.

According to the Kaggle score, **CatBoost** has a slight advantage over **LightGBM**. However, compared to LightGBM, CatBoost took nearly twice as much time to train. Besides, although CatBoost is known for its ability to handle NaN values automatically, it can only do so for categorical NaNs. For floating NaNs, we still need to convert them to a placeholder value such as -999 or the string "missing". Moreover, the output of CatBoost has a low precision, meaning that among the transactions considered fraudulent, only a small part of them are really fraudulent.

The authors stressed that **fraud detection is an ongoing challenge requiring continuous improvement** to adapt to evolving fraud tactics. They recommended **regularly updating models with new data and refining feature engineering techniques** to maintain and improve effectiveness. Furthermore, **collaboration with domain experts and the application of advanced machine learning techniques** were suggested for further advancements.

7 Future Work

Future research directions include:

- **Exploring additional feature engineering techniques** to uncover more intricate patterns.
- **Incorporating real-time data** to improve the detection of new fraud patterns.

- **Investigating ensemble methods and hybrid models** to potentially achieve better performance and robustness.

8 References

1. IEEE-CIS. (2019). IEEE-CIS Fraud Detection. Kaggle. Retrieved February 20, 2025, from <https://www.kaggle.com/competitions/ieee-fraud-detection>